

Intelligent Multi-Class Natural Language Text Sentiment Classifier

AI/ML Internship — Task 2 Report

Task	Task 2 — Sentiment Classification
Domain	Natural Language Processing (NLP)
Programming Language	Python
Framework / Libraries	Scikit-Learn, NLTK, Pandas
Model	Logistic Regression
Feature Extraction	TF-IDF
Classes	Negative, Neutral, Positive

Project objective: Develop a multi-class NLP sentiment classifier that preprocesses natural language text, converts it into numerical features using TF-IDF, trains a classification model, and evaluates its performance using F1-score metrics.

1. Introduction

This project implements an end-to-end Natural Language Processing pipeline for classifying text into three sentiment categories: negative, neutral, and positive. The implementation uses NLTK for text preprocessing, TF-IDF for feature extraction, and Logistic Regression as the classification algorithm.

The system also includes separate training, evaluation, and prediction scripts. This separation makes the project easier to understand, test, maintain, and reuse.

2. Dataset

The project uses the Multiclass Sentiment Analysis Dataset. The dataset was stored locally in `data/dataset.csv` so that the training pipeline is self-contained and reproducible.

Property	Value
Total samples	31,232
Columns	4
Sentiment classes	Negative, Neutral, Positive
Training samples	24,985
Testing samples	6,247

3. Text Preprocessing

The preprocessing pipeline is implemented in `src/preprocessing.py`. The following operations are applied to the input text:

1. Lowercasing
2. Punctuation removal
3. Tokenization
4. Stopword removal
5. Part-of-Speech (POS) tagging
6. WordNet POS mapping
7. POS-aware lemmatization

For example, the sentence "I absolutely LOVED this movie!!!" is transformed into "absolutely love movie" after preprocessing.

4. TF-IDF Feature Extraction

After preprocessing, the cleaned text is converted into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF). The vectorizer is fitted only on the training text and then used to transform the test text. This prevents information from the test set from being used when learning the TF-IDF vocabulary.

Dataset	TF-IDF shape
Training	(24,985, 25,182)

Testing	(6,247, 25,182)
---------	-----------------

5. Model Training

Logistic Regression from Scikit-Learn was used as the multi-class sentiment classifier. The model was configured with `max_iter=1000` and trained using the TF-IDF representation of the training data.

The dataset was divided using an 80/20 train-test split with `random_state=42` and stratification by sentiment label. The trained model, TF-IDF vectorizer, and test data were saved using Joblib.

6. Evaluation

The model was evaluated on the held-out test set using accuracy, precision, recall, F1-score, and a confusion matrix. The following are the verified results from the final local-dataset training run.

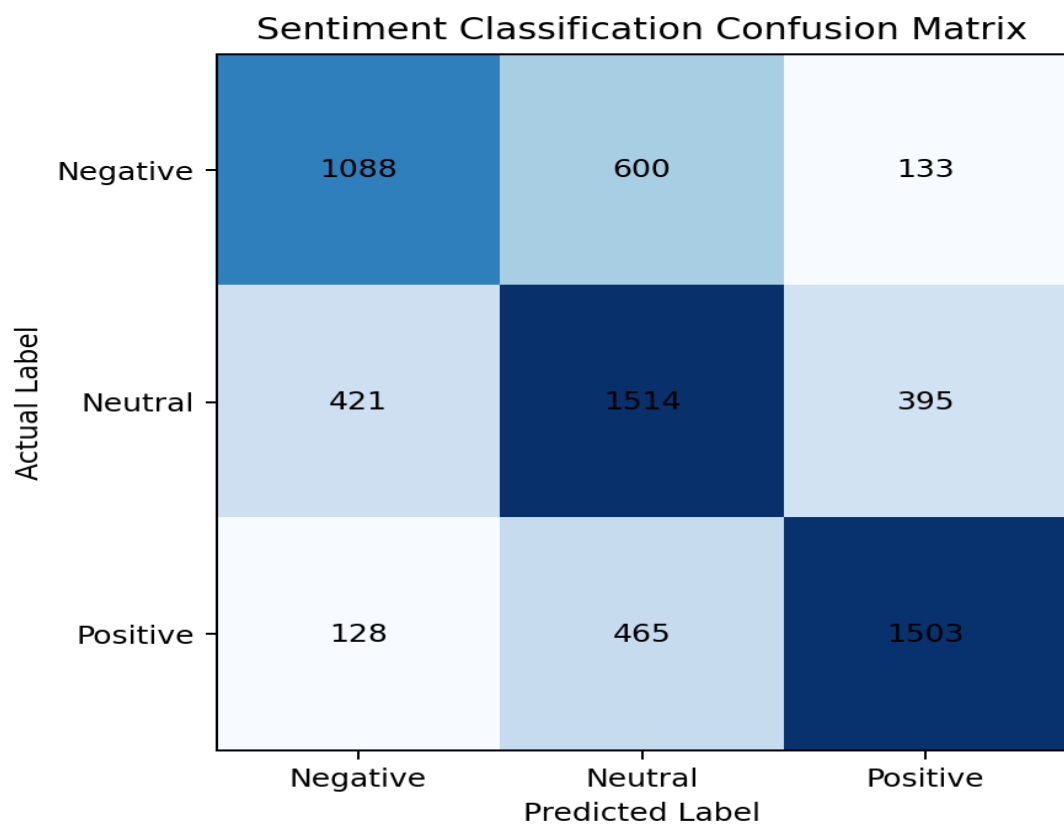
Metric	Result
Accuracy	65.71%
Macro F1-score	0.66
Weighted F1-score	0.66
Negative F1-score	0.63
Neutral F1-score	0.62
Positive F1-score	0.73

Classification report:

Negative — Precision: 0.66, Recall: 0.60, F1: 0.63, Support: 1,821
Neutral — Precision: 0.59, Recall: 0.65, F1: 0.62, Support: 2,330
Positive — Precision: 0.74, Recall: 0.72, F1: 0.73, Support: 2,096

7. Confusion Matrix

The confusion matrix below shows the number of correct and incorrect predictions for each sentiment class. Rows represent actual labels and columns represent predicted labels.



Verified matrix: `[[1088, 600, 133], [421, 1514, 395], [128, 465, 1503]]`.

8. Prediction System

The `src/predict.py` script loads the saved model and TF-IDF vectorizer, accepts a sentence from the user, applies the same preprocessing pipeline, transforms the sentence into TF-IDF features, and returns the predicted sentiment together with class probabilities.

Verified example: For the input "I absolutely love this product!", the model predicted **positive** with the following probabilities:

Sentiment	Probability
Negative	3.17%
Neutral	2.23%
Positive	94.60%

Additional verified test inputs were also classified as follows: "The service was okay, nothing special." → neutral; "This is the worst product I have ever bought." → negative.

9. Project Structure

```
data/dataset.csv
src/preprocessing.py
src/train.py
src/evaluate.py
src/predict.py
models/sentiment_model.pkl
models/tfidf_vectorizer.pkl
models/test_data.pkl
results/classification_report.txt
results/metrics.txt
results/confusion_matrix.png
requirements.txt
README.md
```

10. Conclusion

The project successfully implements a complete multi-class sentiment classification pipeline. It covers text preprocessing with NLTK, TF-IDF feature extraction, Logistic Regression model training, evaluation using F1-score and related metrics, confusion matrix visualization, and prediction of new user-provided text.

The final verified evaluation on the held-out test set produced 65.71% accuracy and a 0.66 macro F1-score. The complete implementation and its generated artifacts are organized into separate data, source, model, results, and documentation components.

11. Technologies Used

Category	Technology
Language	Python
NLP	NLTK
Data Processing	Pandas

Machine Learning	Scikit-Learn
Feature Extraction	TF-IDF
Classifier	Logistic Regression
Model Persistence	Joblib
Visualization	Matplotlib, Seaborn